

Machine Learning – Lecture Notes

Instructor: Dr. Ali Hassan

Date Created: 27 Nov 2016

1 Multi-Layer Neural Network

Consider a supervised learning problem where we have access to labelled training examples $(x^{(i)}, y^{(i)})$. Neural networks give a way of defining a complex, non-linear form of hypotheses $h_\theta(x)$, with parameters $\Theta = [\theta_0 \theta_1 \dots \theta_n]^T$ that we can fit to our data.

To describe neural networks, we will begin by describing the simplest possible neural network, one which comprises a single neuron. We will use the following diagram to denote a single neuron:

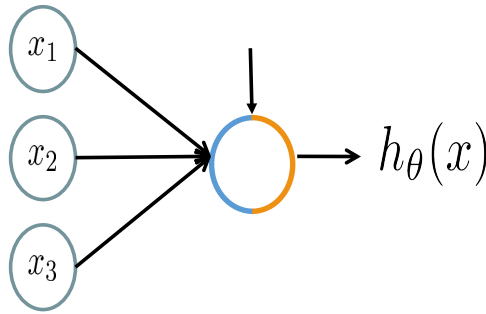


Figure 1: A Single Nueron.

This “neuron” is a computational unit that takes as input x_1, x_2, x_3 (and a +1 intercept or bias term θ_0), and outputs $h_\theta(x) = a(\theta^T x) = a(\sum_{i=1}^3 \theta_i x_i + \theta_0)$, where $a : \Re \mapsto \Re$ is called the **activation function**. In these notes, we will choose $a(\cdot)$ to be the sigmoid function:

$$a(z) = \frac{1}{1 + \exp(-z)}. \quad (1)$$

Thus, our single neuron corresponds exactly to the input-output mapping defined by logistic regression. In this Fig. 1, we will use the first part which is $\theta^T x$ as z (coloured in blue) and refer to it as the pre-activation and will refer the output of the neuron as activation as:

$$a = g(z). \quad (2)$$

Here it is important to notice that if we use a simple linear function as the activation function, then we get the standard linear regression where the output is calculated as:

$$h_\theta(x) = g(\theta^T x), \quad (3)$$

where $g(\cdot)$ is the linear function. However, if we use sigmoid function then the neuron in Fig. 1 turns out to be simple sigmoid function.

Although these notes will use the sigmoid function, it is worth noting that another common choice for f is the hyperbolic tangent, or tanh, function:

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \quad (4)$$

Recent research has found a different activation function, the rectified linear unit (ReLU) function, often works better in practice for deep neural networks. This activation function is different from sigmoid and tanh because it is not bounded or continuously differentiable. The rectified linear activation function is given by,

$$f(z) = \max(0, z). \quad (5)$$

Here are plots of the sigmoid, tanh and rectified linear functions:

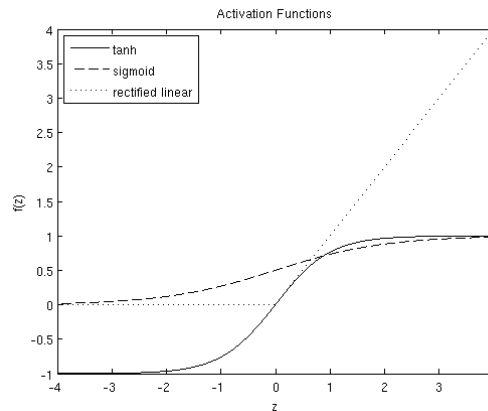


Figure 2: Different Activation Functions.

The $\tanh(z)$ function is a rescaled version of the sigmoid, and its output range is $[-1, 1]$ instead of $[0, 1]$. The rectified linear function is piece-wise linear and saturates at exactly 0 whenever the input z is less than 0.

Finally, one identity that will be useful later: If $f(z) = 1/(1 + \exp(-z))$ is the sigmoid function, then its derivative is given by $f'(z) = f(z)(1 - f(z))$. (If f is the tanh function, then its derivative is given by $f'(z) = 1 - (f(z))^2$.) You can derive this yourself using the definition of the sigmoid (or tanh) function. The rectified linear function has gradient 0 when $z \leq 0$ and 1 otherwise. The gradient is undefined at $z = 0$, though this doesn't cause problems in practice because we average the gradient over many training examples during optimization.

2 Neural Network model and Forward Propagation

A neural network is put together by hooking together many of our simple “neurons”, so that the output of a neuron can be the input of another. For example, here is a small neural network:

In this figure, we have used circles to also denote the inputs to the network. The circles labeled +1 are called **bias units**, and correspond to the intercept term. The leftmost layer of the network is called the **input**

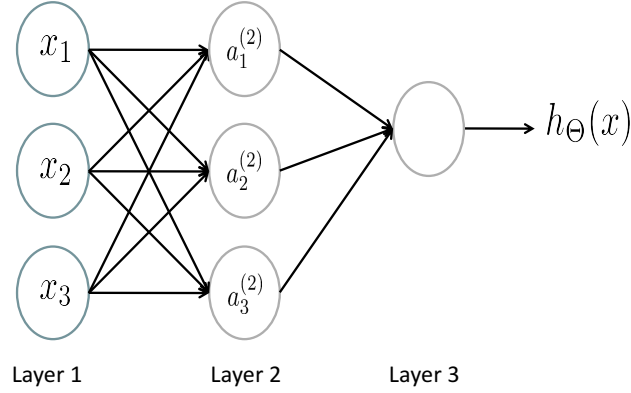


Figure 3: Multi-layer Neural Network.

layer, and the rightmost layer the **output layer** (which, in this example, has only one node). The middle layer of nodes is called the **hidden layer**, because its values are not observed in the training set. We also say that our example neural network has **3 input units** (not counting the bias unit), **3 hidden units**, and **1 output unit**.

We will let n_l denote the number of layers in our network; thus $n_l = 3$ in our example. We label layer l as L_l , so layer L_1 is the input layer, and layer L_{n_l} the output layer. Our neural network has parameters $\Theta = [\theta^{(1)}, \theta^{(2)}]$, where we write $\theta_{ji}^{(l)}$ to denote the parameter (or weight) associated with the connection between j^{th} unit in layer l , and i^{th} neuron in layer $l + 1$. (Note the order of the indices.) Also, $\theta_{i,0}^{(1)}$ is the bias associated with using i in layer $l + 1$. Thus, in our example, we have $\theta^{(1)} \in \mathbb{R}^{3 \times 4}$, and $\theta^{(2)} \in \mathbb{R}^{1 \times 4}$. Note that bias units don't have inputs or connections going into them, since they always output the value +1. We also let s_j denote the number of nodes in layer j . In generic terms we can write that if network has s_j units in layer j , and s_{j+1} units in $j + 1$ layer, then $\theta^{(j)}$ will be of dimension $s_{j+1} \times s_j + 1$.

We will write $a_i^{(l)}$ to denote the **activation** (meaning output value) of unit i in layer l . For $l = 1$, we also use $a_i^{(1)} = x_i$ to denote the i -th input. Given a fixed setting of the parameters θ , our neural network defines a hypothesis $h_\theta(x)$ that outputs a real number. Specifically, the pre-activation z computation of layer 2 is given by:

$$\begin{aligned} z_1^{(2)} &= \theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \\ z_2^{(2)} &= \theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 \\ z_3^{(2)} &= \theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3 \end{aligned} \tag{6}$$

(7)

and the activation of layer 2 are given by

$$\begin{aligned} a_1^{(2)} &= g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3) \\ a_2^{(2)} &= g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3) \\ a_3^{(2)} &= g(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3) \end{aligned} \tag{8}$$

and the pre-activation of layer 3 is given by

$$z_1^{(3)} = \theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)}, \quad (9)$$

and the activation of the output layer is given by

$$a_1^{(3)} = g(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)}), \quad (10)$$

where this is also the output of the neural network $h_\theta(x)$. Note that the above equations can easily be vectorised into a much more compact notation. Specifically, if we extend the activation function $g(\cdot)$ to apply to vectors in an element-wise fashion (i.e., $g([z_1, z_2, z_3]) = [g(z_1), g(z_2), g(z_3)]$), then we can write the equations above more compactly as:

$$z^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix} \quad (11)$$

$$z^{(2)} = \Theta^{(1)} x, \quad (12)$$

where

$$\Theta^{(1)} = \begin{bmatrix} \theta_{10}^{(1)} & \theta_{11}^{(1)} & \theta_{12}^{(1)} & \theta_{13}^{(1)} \\ \theta_{20}^{(1)} & \theta_{21}^{(1)} & \theta_{22}^{(1)} & \theta_{23}^{(1)} \\ \theta_{30}^{(1)} & \theta_{31}^{(1)} & \theta_{32}^{(1)} & \theta_{33}^{(1)} \end{bmatrix} \quad (13)$$

and

$$x = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}. \quad (14)$$

Now we can define the activation functions as:

$$a^{(2)} = g(z^{(2)}) \quad (15)$$

Using the above notation we can define the pre-activations and activations of the next and final layer as:

$$z^{(3)} = \Theta^{(2)} a^{(2)} \quad (16)$$

$$h_\theta(x) = a^{(3)} = g(z^{(3)}) \quad (17)$$

which gives us the output of the neural network if know the values of Θ . We call this step **forward propagation**.

More generally, recalling that we can also use:

$$a^{(1)} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad (18)$$

to denote the values from the input layer, then given layer l 's activations $a^{(l)}$, we can compute layer $l + 1$'s activations $a^{(l+1)}$ as:

$$z^{(l+1)} = \Theta^{(l)} a^{(l)} \quad (19)$$

$$a^{(l+1)} = g(z^{(l+1)}) \quad (20)$$

By organizing our parameters in matrices and using matrix-vector operations, we can take advantage of fast linear algebra routines to quickly perform calculations in our network. Now we only need to figure out how to calculate Θ 's which we will learn in our next lecture.

3 Multi Class Neural Network

To extend the neural network from binary to multi-class we will be using the one vs rest approach in which we will create K classifiers to differentiate K different classes. In the NN case, we will have K neurons/nodes in the output layer. As an example, consider the example shown in Fig. 4 in which we have to classify digits from 0 – 9, then we will have 10 different nodes in the output layer. Consider that we take the images of numbers from the MNIST dataset which contains 28×28 pixels for each character. All 784 pixels are input to the NN in the input layer. These are passed on to the hidden layer and then the output layer which contains 10 neurons in the output layer.

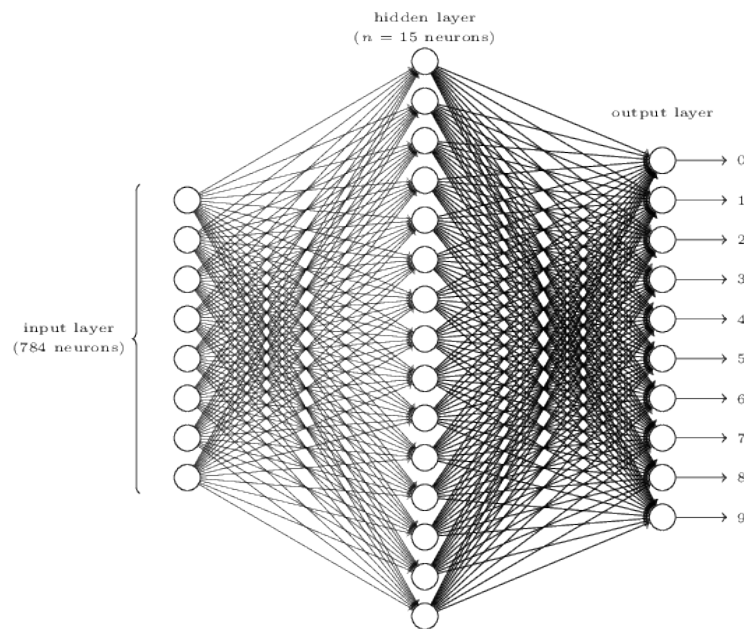


Figure 4: Multi-class Neural Network.

A seemingly natural way of doing that is to use just 4 output neurons, treating each neuron as taking on a binary value, depending on whether the neuron's output is closer to 0 or to 1. Four neurons are enough to encode the answer, since $2^4 = 16$ is more than the 10 possible values for the input digit. Why should our network use 10 neurons instead? Isn't that inefficient? The ultimate justification is empirical: we can try out both network designs, and it turns out that, for this particular problem, the network with 10 output neurons learns to recognize digits better than the network with 4 output neurons.

Suppose we have a four class computer vision problem in which we have to identify a **pedestrian**, **car**, **motorcycle** and a **truck**. Therefore, for a four class problem, we will learn four different hypothesis $h_{\theta}^{(1)}(x)$, $h_{\theta}^{(2)}(x)$, $h_{\theta}^{(3)}(x)$ and $h_{\theta}^{(4)}(x)$. Each neuron will make a prediction between 0 – 1 and we can write the output of the final layer as:

$$h_{\Theta}(x) \approx \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \text{ if } x = \text{Pedestrian.} \quad (21)$$

Similarly,

$$h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \text{ if } x = \text{Car,} \quad (22)$$

$$h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \text{ if } x = \text{MotorCycle,} \quad (23)$$

and

$$h_{\Theta}(x) \approx \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \text{ if } x = \text{Truck.} \quad (24)$$

and in the end we will make the decision based upon the function:

$$\max_i h_{\theta}^{(i)}(x), \quad (25)$$

which will be the output of the NN. One thing to remember is that for the neural network to work properly, we need to transform the input labels.

Consider that we have the input in the following form $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})$. For the four class instead of using $y = \begin{bmatrix} 0 \\ 1 \\ 2 \\ 3 \end{bmatrix}$, we use $y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ for Pedestrian, $y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$ for Car and so on. So, for K classifiers, we have $y \in \mathbb{R}^K$.

4 Back Propagation – NN Cost Function

As we are using the logistic function, so we will be using the cost function which is similar to cost of function of the logistic function. The cost function of the logistic function is give by:

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2 \quad (26)$$

Remember for the NN, we have $h_{\Theta}(x) \in \mathbb{R}^K$ for K class problem. Therefore in our cost function we will sum over K classes as well. Here is the cost function for the NN:

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(h_{\theta}(x^{(i)}))_k + (1 - y_k^{(i)}) \log(1 - h_{\theta}(x^{(i)}))_k \right] + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{i=1}^{s_l} \sum_{j=1}^{s_{l+1}} (\Theta_{ji}^{(l)})^2 \quad (27)$$

In the fitting term, the first part of the equation, we have only added the sum over K , rest is similar to logistic cost function. In the regulariser term, we have summer over all layers L . he first term in the definition of $J(\theta)$ is an average error term of logistic function. The second term is a regularization term (also called a **weight decay** term) that tends to decrease the magnitude of the weights, and helps prevent overfitting. (Note: Usually weight decay is not applied to the bias terms $\theta_0^{(l)}$, as reflected in our definition for $J(\theta)$. Applying weight decay to the bias units usually makes only a small difference to the final network, however.

This cost function above is often used both for classification and for regression problems. For classification, we let $y = 0$ or 1 represent the two class labels (recall that the sigmoid activation function outputs values in $[0, 1]$; if we were using a tanh activation function, we would instead use -1 and $+1$ to denote the labels). For regression problems, we first scale our outputs to ensure that they lie in the $[0, 1]$ range (or if we were using a *tanh* activation function, then the $[-1, 1]$ range).

5 Gradient Descent for NN

Now that we have defined the cost function for NN, the next step is to apply the gradient descent on this cost function and find values of Θ that minimise the cost function.

$$\min_{\Theta} J(\Theta) \quad (28)$$

To do that we know that we have to take the derivative of the cost function and put it in the following equation:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \quad (29)$$

so, we will need to calculate $\frac{\partial}{\partial \Theta_{ji}^{(l)}} J(\Theta)$ and our update equation will look like:

$$\theta_{ij}^{(l)} := \theta_{ij}^{(l)} - \alpha \frac{\partial}{\partial \theta_{ij}^{(l)}} J(\theta) \quad (30)$$

5.1 Initial Values of Θ

To do that first we initialise the values of each parameter $\Theta_{ji}^{(l)}$ to a small random value near zeros. We can use the **normal distribution**, $\mathcal{N}(0, \epsilon^2)$ where $\epsilon = 0.01$. In Matlab, we will use the function:

```

1      % initial value of Epsilon
2      Epsilon = 0.01;
3      % This function will create a matrix Theta of dimensions 10x11 containing values from 0—1
4      Theta = rand(10,11)*(2*Epsilon) - Epsilon;
```

Finally, note that it is important to initialize the parameters randomly, rather than to all 0s. If all the parameters start off at identical values, then all the hidden layer units will end up learning the same function of the input (more formally, $\theta_{ij}^{(1)}$ will be the same for all values of i , so that $a_1^{(2)} = a_2^{(2)} = a_3^{(2)} = \dots$ for any input x). The random initialization serves the purpose of **symmetry breaking**.

5.2 Back Propagation

The most difficult task in the neural network is calculating the partial derivatives for each data parameter θ . To do that, we look at the back-propagation algorithm.

The intuition behind the back-propagation algorithm is as follows. Given a training example (x, y) , we will first run a “forward pass” to compute all the activations throughout the network, including the output value of the hypothesis $h_\theta(x)$. To run this first we run the forward propagation for a four layer NN as follows:

$$\begin{aligned}
 a^{(1)} &= x \\
 z^{(2)} &= \Theta^{(1)} a^{(1)} \\
 a^{(2)} &= g(z^{(2)}) \text{ add } a_0^{(2)} \\
 z^{(3)} &= \Theta^{(2)} a^{(2)} \\
 a^{(3)} &= g(z^{(3)}) \text{ add } a_0^{(3)} \\
 z^{(4)} &= \Theta^{(3)} a^{(3)} \\
 a^{(4)} &= g(z^{(4)}) \text{ add } a_0^{(4)}
 \end{aligned} \tag{31}$$

Then, for each node i in layer l , we would like to compute an “error term” $\delta_i^{(l)}$ that measures how much that node was “responsible” for any errors in our output. For an output node, we can directly measure the difference between the networks activation and the true target value, and use that to define $\delta_i^{(n_l)}$ (where layer n_l is the output layer). How about hidden units? For those, we will compute $\delta_i^{(l)}$ based on a weighted average of the error terms of the nodes that uses $a_i^{(l)}$ as an input. In detail, here is the back propagation algorithm in vectorised form. Remember that $g'(\cdot)$ are the element-wise activations of the pre-activation vectors, so that $g'([z_1, z_2, z_3]) = [g'(z_1), g'(z_2), g'(z_3)]$. We will also use $\bullet$ to denote the element-wise product operator (denoted $.*$ in Matlab, and also called the Hadamard product), so that if $a = b \bullet c$, then $a_i = b_i c_i$ for each element in vector $\mathbf{a}$. So the Back propagation algorithm involves following steps:

1. Perform a feedforward pass, computing the activations for layers L_2, L_3 , and so on up to the output layer L_{n_l} .
2. For each output unit i in layer n_l (the output layer), set

$$\delta^{(n_l)} = (a^{(n_l)} - y) \tag{32}$$

3. For $l = n_l - 1, n_l - 2, n_l - 3, \dots, 2$, (all other layers except the output and input layer) set

$$\delta^{(l)} = \left((\theta^{(l)})^T \delta^{(l+1)} \right) \bullet g'(z^{(l)}) \quad (33)$$

$$\delta^{(l)} = \left((\theta^{(l)})^T \delta^{(l+1)} \right) \bullet (a^{(l)}(1 - a^{(l)})) \quad (34)$$

remember $g'(z) = g(z) \bullet (1 - g(z))$. **Note** that we are not going to calculate the error for the first i.e., input layer.

4. Compute the desired partial derivatives:

$$\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = a_j^{(l)} \delta_i^{(l+1)} \quad (35)$$

OR in the vectorised form, we can do the following:

$$\frac{\partial}{\partial \Theta^{(l)}} J(\Theta) = (a^{(l)})^T \delta^{(l+1)}, \quad (36)$$

to get the error of each layer (l) and now we can update the *theta* in the gradient descent.

Finally, we are ready to describe the full gradient descent algorithm. We implement one iteration of batch gradient descent as follows:

1. Set $\Delta_{ij}^{(l)} = 0$ for all j, i, l .
2. for $i = 1$ to m ,
 - (a) Set $a^{(1)} = x^{(i)}$
 - (b) Perform forward propagation to calculate activations $a^{(l)}$ of each layer $l = 2, 3, \dots, L$
 - (c) Using $y(i)$, calculate $\delta^{(L)} = (a^{(L)} - y^{(i)})$.
 - (d) Compute $\delta^{(L-1)}, \delta^{(L-2)}, \dots, \delta^{(2)}$ using

$$\delta^{(l)} = \left((\theta^{(l)})^T \delta^{(l+1)} \right) \bullet (a^{(l)}(1 - a^{(l)})) \quad (37)$$

- (e) Use $\Delta_{ij}^{(l)} := \Delta_{ij}^{(l)} + a_j^{(l)} \delta_i^{(l)}$

- 3.

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} + \lambda \Theta_{ij}^{(l)} \text{ if } j \neq 0 \quad (38)$$

$$D_{ij}^{(l)} := \frac{1}{m} \Delta_{ij}^{(l)} \text{ if } j = 0 \quad (39)$$

4. Calculated the partial derivative $\frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) = D_{ij}^{(l)}$.

5. Take the step in Gradient descent

$$\Theta^{(l)} := \Theta^{(l)} - \alpha \frac{\partial}{\partial \Theta_{ij}^{(l)}} J(\Theta) \quad (40)$$

We can repeat the above process till the time the gradient descent algorithm converges.